Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Ahmed Gouda Ismail[1*], Mohamed S. Mohamed[1] and Tarek Ahmed Mahmoud[2]

[1*]Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.
[2]Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com
Contributing authors: mohamedms@mtc.edu.eg; tarek.mohamed@eui.edu.eg

Abstract

Multi-object tracking systems usually have two parts: a detector, which finds objects in each frame and gives each a confidence score, and a tracker, which links them over time using fixed score thresholds. When the detector is replaced, retrained, or compressed, its scores change, but the tracker keeps thresholds set for the old detector. This paper presents a training-free self-calibrating control layer (SCL) placed between an unchanged detector and tracker (the host). SCL splits the scores of recent frames with Otsu's histogram threshold and decides whether they are clean or noisy. On clean scores it keeps the host's settings; on noisy scores it chooses which detections reach the host, rescores them, and sets its thresholds. It also continues tracks for trackers without such a step. It needs no labels or future frames. One fixed configuration was tested on seven published trackers, four detection sources, and three datasets. Compared with the tracker alone, it raised the higher order tracking accuracy (HOTA) of two single-stage trackers by 0.9%: OC-SORT, used during development, and PD-SORT, which was not. When all detector scores were halved, two trackers failed completely; with SCL, both recovered at least 97% of their original HOTA. With a noisy RT-DETR-L detector, it raised the multiple object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 12.6% and 16.3% and cut their identity switches by 43% and 47%. On four trackers that already fit their detections, HOTA changed by less than 0.1%. SCL takes 2.95 ms per frame on an ordinary processor.

Keywords: Multi-object tracking, Tracking by detection, Detector confidence calibration, Online adaptation, Otsu thresholding, Real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video, such as people or cars, and gives each object one identity number (ID) that stays the same over time. It is used in surveillance, driving, and drone systems [1, 2], and many of these systems must give results with little delay. Most trackers follow the tracking-by-detection design [3]. First, an object detector, a neural network, looks at each frame and returns candidate boxes. Each box has a confidence score between 0 and 1 that says how sure the detector is that the box holds an object. Then a tracker links the boxes of consecutive frames into tracks, one track per object [4–6]. The tracker uses a box only if its score passes a fixed limit, called a threshold.

Table 1 explains the main terms of this paper in plain words.

Trackers use their thresholds in one of two ways. A two-stage tracker, such as ByteTrack, first matches the high-score boxes to its tracks and then uses lower-score boxes in a second step, only to continue tracks that are still unmatched [5]. A single-stage tracker, such as OC-SORT, uses one threshold and ignores every box below it [6].

In real systems, the detector and the tracker are rarely built together. A detector is replaced by a faster model, retrained for a new camera, or made smaller (quantized) for embedded hardware. The tracker, however, keeps the thresholds that its authors set for another detector. Detector scores are not calibrated across models and conditions [7]: the same object can get a high score from one detector and a much lower score from another. A new detector therefore usually gives scores on a different scale, and nothing in the system notices the change.

The effect can be large. In this paper, halving the scores of a published detector reduces two well-known trackers to 0 higher order tracking accuracy (HOTA) [8], the main tracking metric of this paper, because no detection passes their fixed thresholds (Sect. 7.3). Retuning every tracker for every detector needs labels from the new setting, that is, boxes and identities drawn by people, and a working system usually does not have them.

Current methods solve only part of this problem. Trackers with adaptive thresholds change their own thresholds from the detections of each frame, but each method is built into one tracker [9–11]. Score calibration methods correct the detector scores, but they need labelled data [7]. Methods that adapt a vision system while it runs learn or measure their settings for one model [12, 13]. What is missing is a component that needs no training, works outside the tracker, adjusts itself from the detector's own scores, and leaves the tracker alone when it already fits those scores.

This paper presents such a component: a self-calibrating control layer (SCL) that sits between a detector and a tracker that are both kept fixed (frozen). We call the tracker that SCL serves the *host*, because SCL sits in front of it and works for it without changing its code. The host gives SCL four numbers from its published settings, which we call the host contract (Sect. 4.5). The main idea is simple: the detector's own scores show whether its output fits the host. SCL looks at the scores of recent frames and checks how clearly they separate confident objects from clutter, that is, boxes that are not real objects. If the separation is clear, the host keeps its own settings, and the layer only adds help that the host lacks, such as a step that continues tracks. If it is not clear, SCL chooses which candidates reach the host, gives them new scores, and sets the host's thresholds. The novelty is therefore a layer outside the tracker that needs no labels, uses one fixed configuration for many trackers, and stays out of the way when the tracker already fits the detector.

The results are consistent across trackers, detectors, and data. With halved detector scores, SCL restored two trackers from 0 to about 66 HOTA. It improved every single-stage tracker that was tested, including one that was not seen during design, and it raised HOTA on every MOT17 sequence for both of them. With other detectors, it raised the multiple object tracking accuracy (MOTA), a metric that counts errors, of two-stage trackers by up to 7.9 points on KITTI and by 24.7 points on held-out aerial data, that is, data that was not used during the design. It also cut the identity switches, the cases where an object's ID changes, by 43% and 47% for the two-stage trackers on KITTI and by 45% and 56% on the aerial data. On the same scores, it reached higher HOTA than simple adaptive thresholds by 2.2 to 8.6 points, and it was ahead of the published adaptive-threshold method ACT under halved scores (by 2.0 and 10.1 HOTA points) and on held-out aerial data (by 1.8 HOTA points with YOLOv8n and 22.7 MOTA points with RT-DETR-L). Strong trackers that already fit their detector kept their accuracy, and no statistically significant HOTA loss was found in any tested combination of data, detector, and tracker. The layer takes about 3 ms per frame on an ordinary processor (central processing unit, CPU).

The main contributions are:

1. SCL, a control layer that needs no training, adjusts itself from the detector scores, uses no future frames, keeps the tracker's settings when the scores are clean, and takes full control only when they are noisy (Sect. 4).
2. A host contract, a set of four numbers from the tracker's published settings, that lets one fixed configuration work with single-stage

Table 1 Main terms of this paper in plain words.

Term	Meaning in this paper
Detector	A neural network that finds objects in one video frame and returns a box for each of them.
Candidate (detection)	One box returned by the detector, with a class label (for example, car) and a confidence score.
Confidence score	A number between 0 and 1 that says how sure the detector is that the box holds an object. Different detectors use different score scales.
Threshold	A fixed score limit. The tracker uses a candidate only if its score is at or above the limit.
Tracker	A program that links the candidates of consecutive frames into tracks, one track per object.
Track, identity (ID)	The boxes of one object over time, with one ID number that should never change.
Sequence	One video of a dataset.
Stream	The detector output of one video, frame after frame.
Host	The tracker that SCL serves. It runs after SCL with its own published code and settings, which are never changed.
Host contract	Four published settings of the host that SCL reads (Sect. 4.5).
Matching and birth thresholds	The score a candidate needs to continue an existing track (matching, also called association), and the score it needs to start a new track (birth).
Single-stage, two-stage tracker	A single-stage tracker uses one threshold and ignores every candidate below it. A two-stage tracker also uses low-score candidates in a second step, only to continue existing tracks.
Frozen	Kept fixed: not retrained, not retuned, not changed in any way.
Labels (ground truth)	Boxes and identities drawn by people. They are used only to measure the results, never by SCL.
Development data	Data that was looked at while SCL was designed.
Held-out, predeclared	Data or trackers that were chosen in advance, were not used during the design, and were run once with the frozen SCL.
Logit	The score written on another scale, $\log(s/(1-s))$, Eq. (1). It spreads out scores close to 0 and 1.
Otsu's method	A standard image-processing rule that picks the threshold that best splits a histogram into two groups [14].
HOTA, MOTA, IDF1	Tracking metrics in percent; higher is better (Table 3). HOTA, the main metric, balances finding the objects and keeping their identities.
FP, FN, IDS	False positives (boxes where there is no object), false negatives (missed objects), and identity switches (an object's ID changes). They are counts; lower is better.
Paired comparison	The same tracker runs twice on the same sequences and detections, once alone and once with SCL.
95% interval, significant	The range of likely values of a difference, found by resampling the test sequences 10,000 times (bootstrap). A difference is called significant when this range does not include zero.

and two-stage trackers without changing them (Sect. 4.5).

3. A development path from always-on banding (AOB), an earlier design that always intervened, in which each step is compared with that design (Sect. 5).
4. A test of one fixed configuration on seven published trackers, four detection sources, and three datasets: on data used during the design, and on trackers and data that were chosen in advance and run only once. The test includes statistical intervals for every difference, a comparison with simple adaptive thresholds and with the published ACT method, an analysis of each part of the layer, and a speed measurement on a CPU (Sects. 6 and 7).

The rest of the paper is organized as follows. Section 2 describes the data, the evaluation roles, and the metrics, and Section 3 reviews related work. Section 4 explains SCL and all its settings, and Section 5 shows how it was developed from an earlier design that always intervened. The configuration was then frozen before any external system was run. Section 6 gives the experimental setup, Section 7 the results and their analysis, and Section 8 concludes the paper.

2 Data and Evaluation

2.1 Detectors and data

Table 2 lists the evaluation cells. A cell is one combination of data, detector, and tracker (the trackers are introduced in Sect. 6). Three public datasets are used. MOT17 [15] shows people in city scenes. As is common, each of its training videos is split in time, and the second half, called val-half, is used for evaluation. On MOT17 the trackers use the published detections of the YOLOX-X detector [16], so every tracker sees exactly the same boxes. A detector drops the boxes whose score is below a small value before it passes them on; we call this value the emission floor. The two published MOT17 detection sets have floors of 0.01 and 0.1 (Sect. 7.1). KITTI tracking [17] shows cars and pedestrians seen from a driving car. On its training set the trackers run with two detectors that differ from the MOT17 detector: YOLOv8n [18], a small and fast model, and RT-DETR-L [19], a larger transformer model. UAVDT [20] shows vehicles seen from drones. Its test split, 20 sequences with 16,592 frames, was not used during the design; ByteTrack runs on it with the same two detectors, restricted to their vehicle classes.

To study the score problem directly, the published MOT17 scores were also changed before the tracker by four monotone transforms, which change the score values but keep their order: cubing (s^3), halving ($0.5\,s$), and logit temperatures $T = 2$ and $T = 0.5$, $\sigma(z/T)$, where σ turns a logit back into a score. A temperature divides the logit z of each score by T before turning it back into a score: $T = 2$ pulls all scores toward 0.5, and $T = 0.5$ pushes them toward 0 and 1. These transforms imitate the score changes that a new or retrained detector can cause. The trackers kept their published thresholds. The external systems use the detections released by their authors on MOT17 val-half.

2.2 Evaluation roles and metrics

Every cell has one of two roles (Table 2). Development cells were looked at while SCL was designed: MOT17 with four trackers, KITTI with two other detectors, and the score transforms. The other cells test SCL on trackers and data that it had never seen. The two predeclared external trackers, PD-SORT and Hybrid-SORT, were chosen and named before they were run, and they were run once with the frozen configuration. The held-out UAVDT test split was also run once with the frozen configuration, after a written plan. To check that each external tracker was run correctly, our run of the tracker alone is compared with the number reported by its authors and marked EXACT (within 0.2 HOTA points) or CLOSE (within 1.0 HOTA point, with a known cause that was not tuned).

Table 3 gives the tracking metrics and their definitions. They are built from four counts: false positives (FP), boxes where there is no object; false negatives (FN), objects that were missed; identity switches (IDS), cases where an object's ID changes; and the number of ground-truth boxes (GT). HOTA is the main metric, because it balances finding the objects and keeping their identities. MOTA counts errors and reacts mostly to FP and FN, and the identity F1 score (IDF1) measures how long each object keeps its correct ID. IDS should be read together with FN: a tracker that follows more objects has more chances to switch identities, so IDS can rise while the tracking as a whole becomes better. The metrics are computed with the TrackEval library (commit 12c8791) and the official ground truth of each benchmark, following MOTChallenge practice [21]. On KITTI tracking, HOTA is the official value averaged over cars and pedestrians. On UAVDT, all vehicles form one class, and predictions inside the regions that the dataset marks as ignored are removed.

All comparisons are paired: each tracker runs twice on the same sequences and detections, once alone and once with SCL, and nothing else changes. The difference is computed over all sequences together. To show how certain a difference is, we use a bootstrap [24]: the sequences are drawn again at random, with repetition, 10,000 times (with the fixed random seed 42, so that the result can be repeated), the difference is computed again each time, and the middle 95% of these values gives the 95% interval. A difference is called significant when its interval does not include zero; bold intervals in the tables mark such differences. As a simple check that needs no statistics, we also count the sequences where HOTA rose, stayed the same ($|\Delta\text{HOTA}| < 0.01$), or fell.

HOTA, MOTA, and IDF1 are given in percent, and higher values are better. FP, FN, and IDS are counts, and lower values are better. Runtime is given in milliseconds (ms) per frame, and lower values are better.

3 Related Work

SCL is related to four lines of research: the trackers it works with, the confidence of the detectors that feed them, trackers that adapt their own thresholds, and methods that adapt inference while running. They are reviewed below in this order, and Table 4 compares the closest methods with SCL.

Most practical trackers follow the tracking-by-detection design. SORT predicts where each tracked box will be in the next frame with a Kalman filter, a standard motion predictor, and matches these predictions to the new detections

Table 2 Evaluation cells and their role. A cell is one combination of data, detector, and tracker. Development cells were looked at while the layer was designed; predeclared and held-out cells were chosen in advance and run once with the frozen layer.

Data	Detector	Trackers	Role
MOT17 val-half (7 sequences)	published YOLOX-X detections	ByteTrack (2 settings), OC-SORT, BoostTrack, SparseTrack	development
KITTI tracking training (21 sequences)	RT-DETR-L; YOLOv8n	ByteTrack and BoT-SORT (RT-DETR-L); OC-SORT (YOLOv8n)	development (detector transfer)
MOT17 val-half, score transforms	published YOLOX-X, transformed scores	ByteTrack (2 settings), OC-SORT, BoostTrack	development (robustness)
MOT17 val-half	released PD-SORT / Hybrid-SORT detections	PD-SORT, Hybrid-SORT	predeclared external
UAVDT test (20 sequences)	RT-DETR-L; YOLOv8n	ByteTrack	predeclared held-out
KITTI tracking training, also with halved scores	RT-DETR-L; YOLOv8n	ByteTrack (RT-DETR-L); OC-SORT (YOLOv8n)	development (baselines, ACT, and components)

Table 3 Tracking metrics used in this paper. GT is the number of ground-truth boxes; FP, FN, and IDS are the false positives, missed objects (false negatives), and identity switches, summed over all frames. For IDF1, each ground-truth track is first matched to at most one output track, and IDTP, IDFP, and IDFN are the true-positive, false-positive, and false-negative boxes under this matching. For HOTA, at a localization threshold α, TP_α, FN_α, and FP_α are the matched, missed, and unmatched boxes, and $\mathcal{A}(c)$ is the association accuracy of a matched pair c. HOTA, MOTA, and IDF1 are given in percent.

Metric	Definition	What it measures
FP, FN, IDS	counts summed over all frames	FP: boxes where there is no object; FN: objects that were missed; IDS: times an object's ID changes. Lower is better; IDS is read together with FN, because a tracker that follows more objects has more chances to switch IDs
MOTA [22]	$\mathrm{MOTA} = 1 - \dfrac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}$	one minus the share of errors; reacts mostly to false positives and missed objects
IDF1 [23]	$\mathrm{IDF1} = \dfrac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}$	how long each object keeps its correct ID
HOTA [8]	$\mathrm{HOTA} = \dfrac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha\, \mathrm{AssA}_\alpha}$, $\mathrm{DetA}_\alpha = \dfrac{\lvert\mathrm{TP}_\alpha\rvert}{\lvert\mathrm{TP}_\alpha\rvert + \lvert\mathrm{FN}_\alpha\rvert + \lvert\mathrm{FP}_\alpha\rvert}$, $\mathrm{AssA}_\alpha = \dfrac{1}{\lvert\mathrm{TP}_\alpha\rvert} \sum_{c \in \mathrm{TP}_\alpha} \mathcal{A}(c)$, with $\alpha = 0.05, 0.10, \ldots, 0.95$	balance of finding the objects (detection accuracy, DetA) and keeping their identities (association accuracy, AssA); the main metric of this paper

by their overlap, using the Hungarian method, which finds the best one-to-one matching [4]. ByteTrack added a second matching step that uses low-score detections to continue existing tracks [5], and many later trackers kept this two-stage design. Others, such as OC-SORT [6], use only one score threshold. All these trackers read detector scores through fixed thresholds that were tuned for one detector, usually YOLOX [16]. Seven published trackers of both kinds are used in this paper as hosts, that is, as the trackers that SCL serves, and they are not changed (Sect. 6). A recent survey groups the tracking-by-detection methods into joint detection and embedding, heuristic, motion-based, affinity-learning, and offline methods, and compares them across benchmarks [3].

Many recent trackers improve their association with parts that are trained or designed for a domain: small re-identification networks, which compare the appearance of objects [25], learnable graph matching [26, 27], result-guided feature extraction for faster tracking [28], motion models for low-frame-rate surface-vehicle videos [29], and image restoration by blind super-resolution before tracking in adverse weather [30]. These methods change or retrain the tracker; SCL keeps both the tracker and the detector unchanged and acts only on what passes between them.

These fixed thresholds are a weak point, because scores do not mean the same thing for every detector. Detectors of different families, such as YOLOX [16], YOLOv8 [18], YOLOv9 [31], DETR [32], and RT-DETR [19], give different score distributions. Their confidence is not calibrated across models and conditions, and calibration methods for object detection learn a mapping from labelled data [7]. A tracker tuned for one detector therefore cannot assume that another detector's scores mean the same thing, and labels to recalibrate them are rarely available after deployment. Image processing offers a tool that needs no labels: Otsu's method picks a threshold from a histogram so that the two resulting groups are as separate as possible [14]. SCL applies it to the logits (log-odds) of the detector scores of recent frames.

Some trackers already adapt their own thresholds. Adaptive ByteTrack sets the score threshold of ByteTrack from the detections of each frame [9]. AdapTrack adapts the thresholds of its matching step [10]. ACT sets the high and low thresholds of each frame from the first quartile of the scores and the number of candidates [11]. Other recent trackers use the score or the spread of each detection inside the tracker. ConfTrack weights the Kalman update, the Kalman noise, and the matching cost by the score of each box [33]. UTrack measures the spread of the boxes that non-maximum suppression (NMS), the detector step that removes overlapping boxes of the same object, merges, and uses it as the Kalman noise, so it needs the boxes before this step [34]. PlugTrack blends the Kalman filter with learned motion predictors through a small network trained with labels [35]. These methods are built into the tracker, use only the current frame or each box alone, or need extra trained parts or the inside of the detector. SCL differs in four ways. It uses past frames. It decides for the whole video whether to act at all, and otherwise keeps the tracker's own settings. It works through a host contract, four published settings of each tracker, on several published trackers without changing them. And it is tested as one frozen configuration under score changes and on external systems.

A wider group of methods adapts inference itself. Dynamic neural networks adapt their computation to each input [12], and Chameleon measures again, from time to time, which settings of a video analysis system work best [13]. These methods learn or measure their settings for each model. Our earlier work, AC-MOT (a manuscript submitted for publication), is a controller that needs no training and sets the detector operating point (score threshold, NMS threshold, and input resolution) while running, from image and track cues. Such a controller is tied to the detector it was calibrated with, because a change in the detector's score scale does not change the image. SCL reads the scores instead and acts on the tracker's input and thresholds.

In summary, current methods either adapt thresholds or use the score of each box inside one tracker, calibrate scores with labelled data, or learn their settings for one model. What is missing is a layer that needs no training, adjusts itself from the detector's own scores, works with many unchanged trackers, and keeps a well-matched tracker at its own accuracy. SCL is designed to fill this gap. Table 4 summarizes the differences.

4 Method

SCL is a small step that runs once per frame between the detector and the tracker (Fig. 1). It does four things. First, it looks at the detector scores of the last few frames and puts the candidates into three groups: background, uncertain, and confident (Sect. 4.2). Second, it uses these groups to decide whether the detector output is clean or noisy (Sect. 4.3). Third, it acts on this decision: on clean scores it keeps the tracker's own settings, and on noisy scores it chooses which candidates reach the tracker, assigns them new scores, and sets the tracker's thresholds. Fourth, in both cases it can pass a weak candidate that clearly continues an existing track (Sect. 4.4). The tracker then runs as usual on what it receives. The detector and the tracker are never changed, and Sect. 4.6 lists every setting of the layer.

Table 4 Comparison of SCL with recent adaptive methods for tracking (BT: ByteTrack; ReID: re-identification network; NMS: non-maximum suppression).

Method	Year	Adapted	Statistics	Extra trained part	Detector input	Where it works
Adapt. BT [9]	2023	high/low score split	current frame	none	boxes and scores	inside BT
ConfTrack [33]	2024	Kalman update, noise, and cost	score of each box	ReID	boxes and scores	new tracker
AdapTrack [10]	2024	matching thresholds	current frame	ReID	boxes and scores	new tracker
UTrack [34]	2025	Kalman noise	spread of boxes merged by NMS	none	boxes before NMS	added to existing trackers
ACT [11]	2026	high and low thresholds	current frame	none	boxes and scores	inside the tracker
PlugTrack [35]	2026	mix of Kalman and learned motion	track boxes	network trained with labels	boxes	added to two learned predictors
Küppers [7]	2020	detector scores	labelled data	calibration map	boxes and scores	detector output
Chameleon [13]	2018	pipeline configuration	profiling	none	whole pipeline	analytics pipeline
AC-MOT (ours, submitted)	2026	detector operating point	image and track cues	none	sets its settings	before the detector
SCL (this paper)	–	candidates, scores, thresholds	past frames, no labels	none	boxes and scores	between detector and seven unchanged trackers

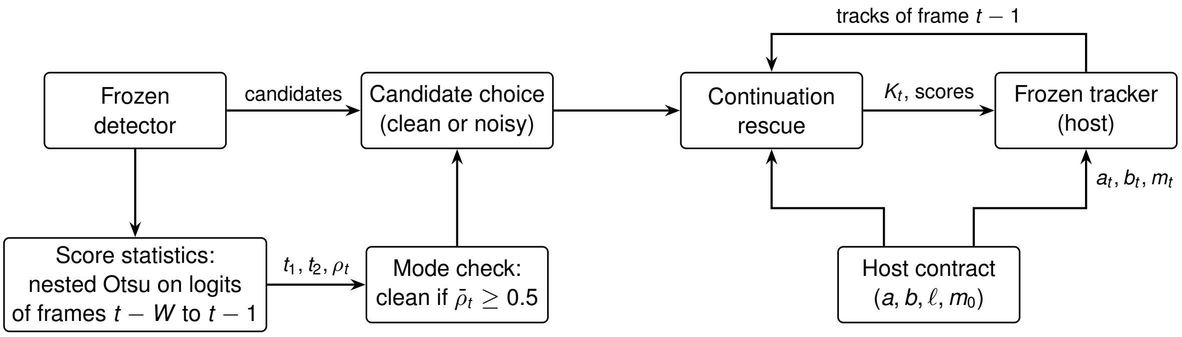

tracks of frame $t-1$
Frozen detector
candidates
Candidate choice (clean or noisy)
Continuation rescue
K_t, scores
Frozen tracker (host)
a_t, b_t, m_t
Score statistics: nested Otsu on logits of frames $t-W$ to $t-1$
t_1, t_2, ρ_t
Mode check: clean if $\bar{\rho}_t \geq 0.5$
Host contract (a, b, ℓ, m_0)

Fig. 1 SCL between a frozen detector and a frozen tracker (the host). Boxes are processing steps and arrows show what each step passes on. The score statistics use earlier frames only. The host contract holds the tracker's published settings. The tracker receives the chosen candidates K_t, their scores, and its thresholds a_t, b_t and tolerance m_t for the frame

4.1 Inputs and outputs

In each frame t, the detector gives a list of candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$: a box $\mathbf{x}_i$, a score $s_i \in (0,1)$, and a class label c_i for each one, down to a lowest score called the emission floor. The layer does not use the score itself but its logit (log-odds), which spreads out scores close to 0 and 1:

$$z_i = \log \frac{s_i}{1-s_i}. \tag{1}$$

For example, a score of 0.5 gives $z_i = 0$, a score of 0.9 gives $z_i \approx 2.2$, and a score of 0.1 gives $z_i \approx -2.2$. The logistic function $\sigma(z) = 1/(1+e^{-z})$ turns a logit back into a score. Table 5 lists all symbols of the method.

As defined in Sect. 1, the tracker that the layer serves is called the host. It gives the layer four numbers about itself, the host contract $h = (a, b, \ell, m_0)$. These are simply the settings that its authors published: a is the score a detection needs to be matched to a track in the first step, b is the score it needs to start a new track, ℓ is the lowest score the tracker uses at all, and m_0 is its matching tolerance (one minus the smallest overlap that still counts as a match). Overlap is measured by the intersection over union (IoU)

Table 5 Main symbols of the method.

Symbol	Meaning
t	frame index
D_t	candidates of frame t given by the detector
$\mathbf{x}_i, s_i, c_i$	box, score, and class label of candidate i
z_i	logit (log-odds) of the score s_i, Eq. (1)
$\sigma(\cdot)$	logistic function, turns a logit back into a score
$h = (a, b, \ell, m_0)$	host contract: the tracker's published settings
a, b	score needed to match a track, and to start a new track
ℓ	lowest score the tracker uses
m_0	matching tolerance: one minus the smallest IoU that counts as a match
W	number of past frames used for the statistics (10)
t_1, t_2	first and second Otsu split on the logits
Z_t^W	logits of the last W frames, $t - W$ to $t - 1$
ρ_t, $\bar{\rho}_t$	share of confident logits in Z_t^W, and the median of all ρ values so far
u_i	rank of candidate i among recent scores, from 0 to 1
K_t, $\tilde{s}$	candidates passed to the tracker, and their scores
a_t, b_t, m_t	thresholds and tolerance used by the tracker in frame t
$\mathcal{T}_{t-1}$	tracks of the previous frame
g_t, r_t	camera motion in frame t, and its ratio to the usual motion
IoU	intersection over union: overlap of two boxes, from 0 to 1

of two boxes: the area they share divided by the area they cover together, from 0 (no overlap) to 1 (the same box). A single-stage tracker uses one threshold, so $\ell = \min(a, b)$. A two-stage tracker also uses weaker detections in a second step, so $\ell < \min(a, b)$.

In every frame the layer returns five things: the candidates K_t that the tracker will see, their scores $\tilde{s}$, and the thresholds (a_t, b_t) and tolerance m_t that the tracker will use in this frame:

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t;\, h\right). \quad (2)$$

Here $Z_{<t}$ are the logits of the earlier frames, $\mathcal{T}_{t-1}$ are the tracks of the previous frame, and g_t is an optional measure of camera motion in frame t. In words, Eq. (2) says that the output for frame t depends only on the logits of earlier frames, the candidates of frame t, the tracks of frame $t-1$, the camera motion, and the tracker's contract. The decision for frame t uses only past frames; frame t is added to the statistics after the decision, and no future frame is ever used.

Four goals guided the design. The layer needs no training and no labels. One frozen configuration must work for every tracker, detector, and dataset. When the tracker already fits the detector, its input must stay the same. And the layer must take little time compared with the detector.

4.2 Self-calibration from detector scores

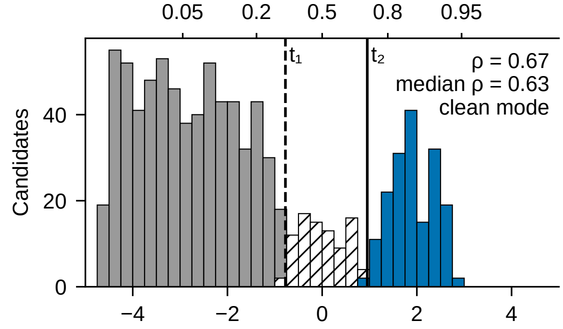
A detector that suits the scene gives many low scores to background and a separate group of high scores to objects. The layer checks, without labels, how clearly this happens. It collects the logits of the candidates of the last $W = 10$ frames that remain after duplicate removal (Sect. 4.3) and splits them twice with Otsu's method [14], which finds the threshold that best separates a histogram into two groups. The first split, t_1, separates background (low logits) from foreground (high logits). The second split, t_2, is found in the same way on the foreground logits only, and divides them into uncertain and confident candidates. We call these two splits nested Otsu splits. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{z \in Z_t^W : z \geq t_2\}\right|}{\left|\{z \in Z_t^W : z \geq t_1\}\right|}, \quad (3)$$

where Z_t^W are the logits of the last W frames, $t - W$ to $t - 1$, the same logits that give t_1 and t_2. It shows how clearly the detector separates objects from clutter. In words, ρ_t is the number of confident candidates divided by the number of foreground candidates in these frames. It is computed before frame t is processed; the candidates of frame t enter the statistics only after the decision for frame t. For example, if 40 candidates are in the foreground and 30 of them are confident, then $\rho_t = 0.75$. A high ρ_t means that most foreground candidates are confident; a low ρ_t means that many of them are uncertain. Because the splits are made on logits, they move with the scores when the whole logit scale is stretched or shifted, for example by a change of temperature, and each group keeps the same candidates. Figure 2 shows the logits and the three groups for a clean stream and for a noisy stream.

These groups only make sense if there is a real background group. Some detectors give very

(a) Clean stream: MOT17, YOLOX-X



(b) Noisy stream: KITTI, RT-DETR-L

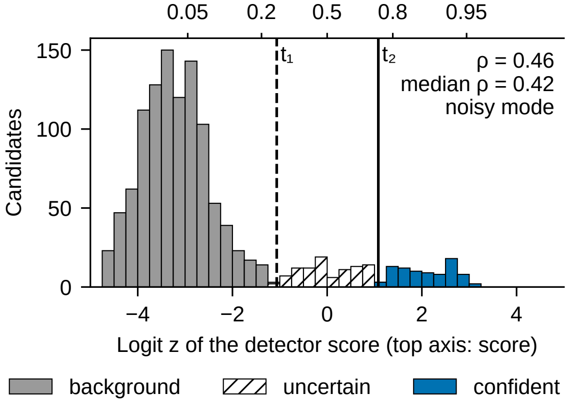


Fig. 2 What the layer sees: the logits of the candidates of the last 10 frames, just before its decision for frame 101 of a sequence, with the two Otsu splits t_1 (dashed line) and t_2 (solid line). Plain light bars: background; hatched bars: uncertain; plain dark bars: confident. The top axis gives the detector score that matches each logit. (a) MOT17-02 with the published YOLOX-X detections: most foreground candidates are confident, and the stream is clean, so the tracker keeps its settings. (b) KITTI sequence 0001 with RT-DETR-L: many candidates are uncertain, and the stream is noisy, so the layer chooses the candidates and sets the thresholds. All values are those of the frozen layer in the runs of Sect. 7

few low scores; then the first split falls inside the objects. The layer detects this case, when fewer candidates lie below t_1 than above it, and counts the frame as clean, $\rho_t = 1$. This background check covers, for example, the first frames of a sequence with only about ten candidates per frame, where both splits would otherwise fall among the objects ($t_1 \approx 0.6$ and $t_2 \approx 0.93$ in probability terms).

4.3 Clean and noisy modes

The mode decides how much the layer may change. A clean stream means that the detector already separates objects from clutter, so the tracker's own settings are kept. A noisy stream means that many candidates are uncertain, so the layer takes over the choice of candidates and thresholds.

Choosing the mode. The stream is clean at frame t if the median $\bar{\rho}_t$ of all ρ values so far (ignoring cold frames) is at least one half, and noisy otherwise. Before the first valid split, for example in the first frame, the mode is called cold. Because the median covers the whole stream so far, the mode describes the detector and the scene, and a short drop in confidence, for example during fast camera motion, does not change it.

Cold and clean modes. The tracker keeps its own thresholds. In the clean mode there is one exception: if a threshold is so high that it would reject the confident group, it is lowered to t_2:

$$a_t = \min(a, \sigma(t_2))\,, \qquad b_t = \min(b, \sigma(t_2))\,, \quad (4)$$

where σ is the logistic function, which turns a logit back into a score. In words, each threshold stays as the tracker set it, unless it is higher than the score $\sigma(t_2)$ at which the confident group begins; then it is lowered to that score. Scores are passed unchanged, and only cross-class duplicates (one box given two class labels) are removed. When the confident group already lies above the tracker's thresholds, the tracker therefore receives exactly its normal input.

Example. When the detector scores are halved, no score is above 0.5. A tracker with $a = 0.6$ and $b = 0.7$, such as ByteTrack in its official setting (Table 6), then receives no detection that can start a track, so it never creates a track. The Otsu splits are computed from the halved scores themselves, so they move with them. In the clean mode, Eq. (4) then lowers the tracker's thresholds to $\sigma(t_2)$, and the confident candidates reach the tracker again. In the noisy mode, the new scores below place them at or above the thresholds of 0.5. Section 7.3 shows the measured effect.

Noisy mode. Here the layer decides which candidates the tracker may use, and for what. Confident candidates ($z \geq t_2$) may start and continue tracks. Uncertain candidates ($t_1 \leq z < t_2$)

may only continue existing tracks. Background candidates ($z < t_1$) are dropped. Each score is replaced by a new one based on its rank u_i among recent scores. These are kept in a small memory, the remap memory: the scores of 20 past frames, one frame taken every 10 frames. The rank is the position of the score in this memory (from 0 for the lowest to 1 for the highest),

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \tag{5}$$

and the tracker's matching and birth thresholds are both set to 0.5, $a_t = b_t = 0.5$. In words, the lowest-ranked confident candidate gets 0.5 and the highest-ranked one gets close to 1, while an uncertain candidate gets a score between 0.1 and 0.5 in the same order. Confident candidates thus get scores of 0.5 and above, and uncertain ones get scores below 0.5, so they can only reach a tracker's low-score step. Duplicates are removed with the help of the tracks: a weaker candidate that overlaps a stronger one with an intersection over union (IoU) above one half is removed, unless a different track of the previous frame claims it. This keeps people who stand close together in a crowd.

How the layer helps identities. An identity switch happens when a track takes the box of another object, or when a lost object is found again and starts a new track with a new ID. The layer is designed to reduce both cases. In the noisy mode it keeps background candidates away from the tracker, so fewer false boxes can be matched to a track. The continuation rescue (Sect. 4.4) keeps a track alive when its object gets low scores for a few frames, so the object keeps its ID. The duplicate rule uses the tracks of the previous frame, so a box that belongs to a different track is not removed and its object is not lost.

4.4 Track continuation and camera motion

A tracker ignores every candidate below its lowest threshold, so a track can be lost when its object gets low scores for a few frames. This matters most for single-stage trackers, which have no low-score step. The rescue gives such a candidate a second chance, but only when it clearly continues an existing track. Three conditions must hold: the candidate is in the foreground ($z \geq t_1$, or any candidate for a single-stage tracker when there is no background group); it overlaps a track of the previous frame with an IoU of at least one half; and no other usable candidate covers that track. Also, the tracker must be unable to see the candidate with the thresholds of this frame. The candidate is then passed with its score raised just above the tracker's lowest threshold. For a two-stage tracker, this only applies to candidates below its low threshold ℓ. For a single-stage tracker, it applies to candidates below the lowest threshold used in this frame, so the rescue gives it the continuation step it lacks.

When the camera moves fast, a box moves more between two frames, and a fixed overlap requirement can break a correct match. In noisy frames, and only when a motion measure is available, the matching tolerance therefore grows with the current camera motion; r_t is the current motion divided by its median over the last 100 frames (used after a warm-up of 5 frames). The motion is the shift between two consecutive grayscale frames at quarter resolution, found by phase correlation, a standard method that measures how far one image is shifted against another [36], and divided by the image diagonal. The tolerance is

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \tag{6}$$

In words, when the camera moves no faster than usual ($r_t \leq 1$), the tolerance stays at m_0. When the motion is twice its usual value ($r_t = 2$), the gap $1 - m_0$ is halved: with $m_0 = 0.8$, the tolerance becomes 0.9, so a match needs an IoU of only 0.1 instead of 0.2. The tolerance never exceeds 0.95.

4.5 Host contract and adapters

Table 6 lists the contracts used in this paper. Each value is taken from the tracker's published configuration; none is fitted. A small piece of code for each tracker family, called an adapter, passes the layer's decision to the tracker: the chosen candidates and their scores replace the detector output of the frame, and the new thresholds and tolerance replace the tracker's own for that frame. Any step that the authors apply after tracking, such as filling short gaps in tracks (interpolation) or linking tracks after the video ends, is kept. The rules and their constants are the same for every tracker. A tracker enters only through its four contract values, and the adapter only passes on the decision; it never makes one.

4.6 Fixed constants

Table 7 lists every constant of the layer; the same values are used for every tracker, detector, and dataset. The thresholds t_1 and t_2 are not constants: they are computed again in every frame. Each constant has a simple origin. The overlap of one half for duplicates and rescue is the standard matching threshold of MOT evaluation, an IoU of 0.5. The score ranges of Eq. (5) put the two groups on the common thresholds of 0.5 (matching and birth) and 0.1 (low score). The cap of 0.95 keeps a minimum overlap of 0.05 for every match. The window, the motion history, and the remap memory are default values that were kept fixed in all development stages.

4.7 Per-frame steps and cost

Algorithm 1 gives the steps for one frame. The work per frame is small: the layer sorts the logits of at most W frames for the two Otsu splits, computes overlaps between candidates and with the tracks of the previous frame, and, when motion is used, computes one phase correlation on a quarter-resolution image. With n candidates per frame and k tracks, the cost is $O(Wn\log(Wn) + n^2 + nk)$ plus a fixed image cost. The three terms come from sorting the logits of the window, comparing every pair of candidates, and comparing every candidate with every track. The layer stores only the logits of the last W frames, the per-frame ρ values, the remap memory, and the last 100 motion values. Table 8 and Fig. 3 give the measured time per frame of a full pipeline on an ordinary processor, with and without SCL.

5 Development of the Method

5.1 Starting point: always-on banding

SCL was developed from an earlier design of the same research program, called always-on banding (AOB). AOB computed the same nested Otsu groups, or score bands, but treated every stream as noisy, so it applied the bands in every frame; this is where its name comes from. In every frame it also removed every weaker candidate that overlapped a stronger one with IoU above one half, and gave all candidates new scores by rank on fixed ranges. It passed no candidate in the first frame. AOB was designed for noisy detector streams, and it is the baseline of this section.

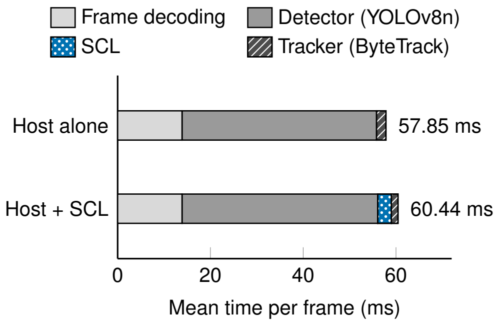


Fig. 3 Mean time per frame of each stage for the pipeline with the tracker alone and with SCL (4-vCPU Intel Xeon, no GPU). Light and medium plain segments: frame decoding and detector; dotted segment: SCL; striped segment: tracker. The labels give the measured end-to-end time

Table 9 compares AOB and SCL with the same strong published trackers on MOT17 [15]. AOB lowered the HOTA of SparseTrack by 4.15 points and that of BoostTrack by 5.89 points online and 5.37 points with BoostTrack's post-processing that fills short gaps in tracks (linear and gradient boosting interpolation, GBI), all with intervals below zero. Two causes were found. First, removing overlapping boxes as duplicates deleted people hidden behind others in crowds. Second, the self-calibrated thresholds were stricter than the trackers' own: for SparseTrack and online BoostTrack, FP fell by 1,630 and 1,351, but FN rose by 4,955 and 6,167. On the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack versions identical to the host. Figure 4 shows the same comparison.

What this stage showed. A layer that acts in every frame harms trackers that already fit the detector: strong published trackers lost between 4.15 and 5.89 HOTA points in all three settings. Such a layer cannot be placed safely in front of an unknown tracker. The next steps therefore let the layer decide from the scores whether to act at all.

Table 6 Host contracts: the published settings of each tracker, as used by the layer.

Tracker (setting)	Type	assoc a	birth b	low ℓ	match m_0	Decision mapped to
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8	thresholds, candidates, scores
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8	same
BoT-SORT (library default)	two-stage	0.25	0.25	0.1	0.8	same
SparseTrack	two-stage	0.6	0.7	0.1	0.85	same
BoostTrack	two-stage	0.6	0.6	0.1	0.7	same
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	same
OC-SORT	single-stage	0.6	0.6	0.6	0.7	same
PD-SORT	single-stage	0.6	0.6	0.6	0.7	same

Table 7 Fixed constants of SCL.

Constant	Value
Statistics window W	10 frames
Mode rule	clean if median $\rho \geq 0.5$ over non-cold frames so far
Background check	$\rho = 1$ if fewer candidates below t_1 than above
Noisy score ranges	primary $[0.5, 1)$, extension $[0.1, 0.5)$
Noisy association and birth thresholds	0.5
Remap memory	20 frames, one every 10 frames
Duplicate and rescue IoU	0.5
Rescued score	lowest used threshold $+0.001$
Motion cue	phase correlation, quarter resolution
Motion history, warm-up	100, 5 frames
Tolerance cap	0.95

Table 8 Baseline comparison of the per-frame time on a 4-vCPU Intel Xeon (2.10 GHz, no GPU) with a live YOLOv8n detector (736 pixels) and ByteTrack on three KITTI sequences (2,279 timed frames per arm). Host alone: the same pipeline without SCL. Mean / 95th percentile in milliseconds.

Stage	Host alone	Host + SCL
Frame decode	13.88 / 16.21	13.88 / 16.51
Detector (YOLOv8n, 736 px)	41.91 / 58.41	42.17 / 59.43
SCL	–	2.95 / 4.39
Tracker (ByteTrack)	2.04 / 3.57	1.43 / 2.64
End to end	57.85 / 76.35	60.44 / 80.78

Table 9 Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections). Host: the tracker alone, without any layer (our run of the published tracker). Δ: tracker with the earlier design AOB or with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ AOB	Δ SCL
SparseTrack [37]			
HOTA	68.88	**−4.15** **[−5.54, −1.66]**	+0.06 [−0.01, +0.25]
MOTA	77.85	**−6.14** **[−8.10, −2.74]**	+0.08 [−0.03, +0.30]
IDF1	81.97	**−4.49** **[−6.19, −1.96]**	+0.16 [−0.02, +0.70]
IDS	124	−16 [−47, +12]	+7 [0, +19]
BoostTrack, online [38]			
HOTA	68.49	**−5.89** **[−7.76, −2.87]**	0 (identical)
MOTA	75.50	**−8.87** **[−12.37, −7.14]**	0 (identical)
IDF1	81.41	**−6.22** **[−9.33, −2.85]**	0 (identical)
IDS	113	**−38** **[−81, −1]**	0 (identical)
BoostTrack, with GBI [38]			
HOTA	71.72	**−5.37** **[−7.30, −2.75]**	0 (identical)
MOTA	81.03	**−8.73** **[−12.59, −6.50]**	0 (identical)
IDF1	84.16	**−5.65** **[−8.96, −2.77]**	0 (identical)
IDS	97	−27 [−73, +11]	0 (identical)

5.2 Design steps

Each step from AOB to SCL was made to remove one cause found before it, and it was compared with the step before it on development data (Table 10). The steps, in the order in which they were made, were the following.

- *Starting point.* Running AOB inside the new framework showed the same problem: it reduced ByteTrack at floor 0.1 from 67.70 to 56.41 HOTA and OC-SORT from 66.44 to 45.72.
- *Mode gate.* The two causes found in Sect. 5.1 were thresholds stricter than the trackers' own and the removal of hidden people as duplicates. The mode gate adds the mode check, the clean

Algorithm 1 SCL for frame t. The window holds the logits of frames $t-W$ to $t-1$, so the decision for frame t uses no future frame.

```
Input: candidates $D_t$, tracks $\mathcal{T}_{t-1}$, motion cue $g_t$,
    contract $(a, b, \ell, m_0)$, constants of Table 7
Output: candidates $K_t$, scores $\tilde{s}$, thresholds
    $a_t, b_t$, tolerance $m_t$
$z_i \leftarrow$ logit of $s_i$ for all $i$    ▷ Eq. (1)
if no valid split exists then    ▷ e.g., first frame
    mode ← cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window    ▷ Sect. 4.2
    $\rho_t \leftarrow$ Eq. (3) with the background check
    mode ← clean if median$(\rho) \geq 0.5$, else noisy
end if
if mode is cold or clean then    ▷ Sect. 4.3
    remove cross-class duplicates; keep raw scores
    if clean then set $(a_t, b_t)$ by Eq. (4)
    end if
else    ▷ noisy mode
    remove duplicates with track context
    drop $z < t_1$; remap scores by Eq. (5)
    $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold)    ▷ Sect. 4.4
$m_t \leftarrow$ Eq. (6) if noisy, else $m_0$
pass $K_t$, $\tilde{s}$, $a_t$, $b_t$, $m_t$ to the tracker
add the logits left after duplicate removal to the window
add $g_t$ to the motion history
store the tracks of frame $t$ after the tracker update
```

mode that keeps the tracker's settings, and the crowd-safe duplicate rule. It restored SparseTrack from 64.72 to 68.93 and BoostTrack from 62.61 to 68.37.

- *Whole-stream mode.* The mode still changed in the middle of some streams. Choosing it from the whole stream removed these changes and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071.
- *Continuation rescue.* Single-stage trackers still lost tracks whose objects got low scores for a few frames. The rescue added 0.2 to 0.4 points for OC-SORT.

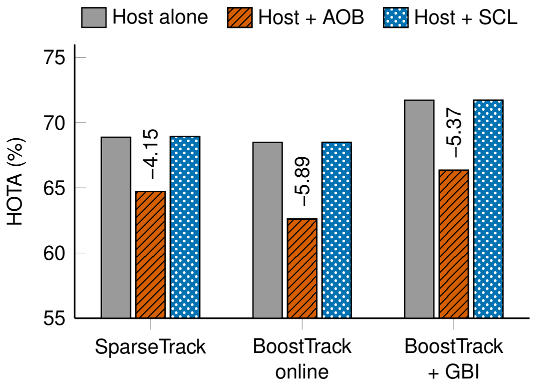


Fig. 4 HOTA on MOT17 val-half of strong published trackers (SparseTrack and BoostTrack in two settings). Plain bars: the tracker alone; striped bars: with AOB; dotted bars: with SCL. The vertical labels give the change caused by AOB in HOTA points

- *Background check.* Streams with few low-score candidates need a separate rule (Sect. 4.2). The background check completed SCL: it returned ByteTrack and BoostTrack to output identical to the host and raised OC-SORT to 67.04 and 66.91 at the two floors.

Four other variants were also tested at these steps and were not kept (Table 10).

What this stage showed. After these steps, the layer kept the strong two-stage trackers at their own accuracy and improved the single-stage OC-SORT. This is the behavior that the frozen configuration was then tested for on new trackers, detectors, and data (Sect. 7).

5.3 Freezing and testing

The final configuration was frozen before any external test. It was saved as a fixed, named version of the code (a version-control tag) and protected by a lock file that stores a fingerprint (cryptographic hash) of the layer, its configuration, and the evaluation scripts, so that any later change would be detected. Every later run checked the lock before running. Sixty-one automated tests check that the layer uses no future data, that its state resets between sequences, that clean streams pass through unchanged, that the output does not depend on detector, tracker, or sequence names, that stretching or shifting all logits leaves the groups unchanged where designed, and that the lock is intact.

Table 10 Design steps from AOB to SCL on development data (HOTA, %), and versions that were tested and rejected. The floor of ByteTrack and OC-SORT is given in parentheses.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)

Rejected version	*Evidence*
no candidates at frame 1	−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage	FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames	ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band	KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

6 Experimental Setup

The experiments test three expectations from the design. A tracker that already fits its detector should keep its accuracy. A tracker that does not fit, because it has no continuation step or because the detector or its score scale changed, should gain. Both should hold on trackers and data that were not seen during design.

Seven published trackers serve as hosts: ByteTrack [5], BoT-SORT [39], OC-SORT [6], SparseTrack [37], BoostTrack [38], PD-SORT [40], and Hybrid-SORT [41]. Each host uses the authors' code and published configuration, and its contract is given in Table 6. ByteTrack is used in two published settings: the official setting, with the thresholds that its authors used on MOT17, and the library setting, with the default thresholds of the widely used Ultralytics library, which also provides the BoT-SORT host.

In every comparison the baseline is the host alone, called Host in the tables: the published tracker without SCL, run by us with its published configuration and the same detections. The second run adds the frozen SCL and changes nothing else. Both runs use the same environment. Where the authors report their own numbers, these are listed next to our reproduction of the host.

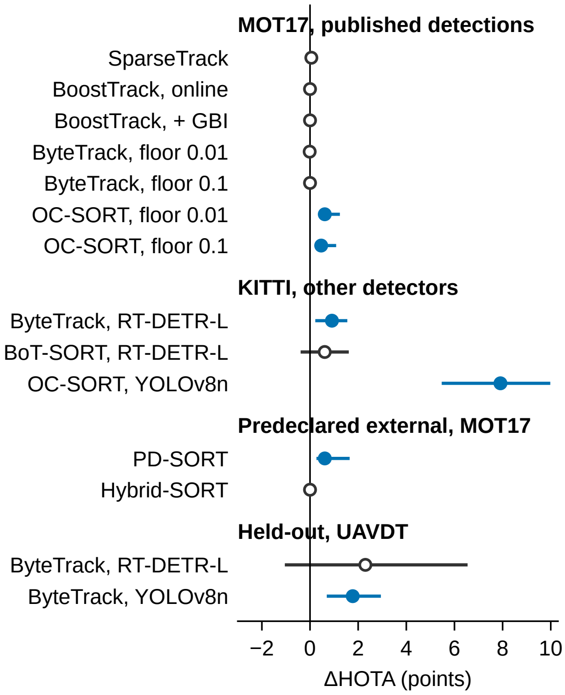


Fig. 5 Difference in HOTA, host with SCL minus host alone, in points, for every tested cell. The circle is the difference and the horizontal line its 95% paired-bootstrap interval. Filled circles: interval above zero; open circles: interval contains zero, or identical output

7 Results and Analysis

Figure 5 shows the HOTA difference between each host with SCL and the host alone for every tested cell. The next subsections give the baseline comparison for each part of the evaluation, in the order of the three expectations of Sect. 6: well-matched trackers, trackers that do not fit their detector, and systems not seen during design.

7.1 Results across trackers

Table 11 compares the development trackers with their hosts on MOT17. The two-stage trackers with well-matched detections stayed at the accuracy of their hosts. BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave exactly the same output as without the layer, and ByteTrack at floor 0.01 and SparseTrack stayed within 0.06 HOTA points of their hosts (−0.014 and +0.055). This is the self-limiting behavior of the clean mode (Sect. 4.3), which the earlier design did not have (Table 9).

The single-stage OC-SORT improved at both emission floors. Its HOTA rose by 0.61 points ($[+0.36, +1.24]$) at floor 0.01 and by 0.46 points ($[+0.27, +1.09]$) at floor 0.1, and its MOTA by 1.27 and 1.26 points. Its identity switches also fell, by 8 at floor 0.01 ($[-15, -2]$) and by 14 at floor 0.1. HOTA was higher on 7 of 7 sequences in both settings (Fig. 6). The gain comes from the continuation rescue. OC-SORT drops every detection below its single threshold, and the layer passes it the foreground candidates that continue a track that would otherwise stay unmatched.

7.2 Results with other detectors

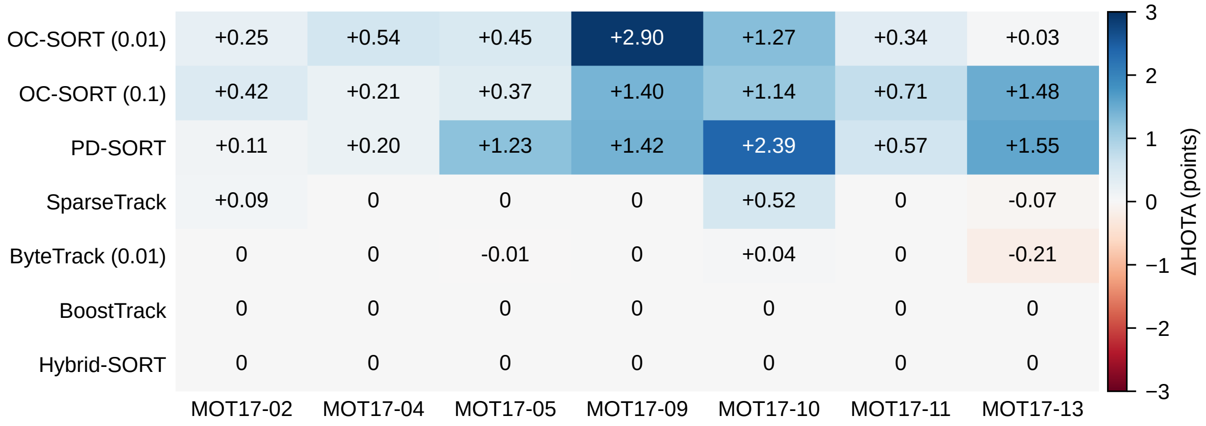
Table 12 reports the KITTI results with the two other detectors. With the noisy RT-DETR-L detector, the two-stage trackers gained in MOTA and IDF1. ByteTrack gained 5.98 MOTA points ($[+1.16, +9.94]$), 3.87 IDF1 points, and 0.91 HOTA points ($[+0.22, +1.55]$). BoT-SORT gained 7.86 MOTA points ($[+2.21, +13.44]$) and 3.10 IDF1 points. Both also had fewer identity switches: 297 fewer for ByteTrack and 226 fewer for BoT-SORT. With YOLOv8n, the single-stage OC-SORT gained 7.91 HOTA points ($[+5.47, +9.98]$), 9.40 MOTA points, and 9.60 IDF1 points. It had 79 more identity switches, because it now followed many more objects: its missed objects (FN) fell from 21,886 to 16,576, and its IDF1, which measures how well identities are kept, rose. Figure 7 shows one frame of this cell: OC-SORT alone tracked 6 of the 10 labelled objects, and with SCL it tracked 9, including parked cars further down the street.

Table 11 Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections). Paper: number reported by the tracker's authors. Host: the tracker alone, without SCL (our run of the published tracker). Δ: host with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. The floor is the lowest detector score emitted. IDS: identity switches, a count (lower is better); –: not reported by the authors.

Metric	Paper	Host	Δ [95% CI]
SparseTrack [37]			
HOTA	69.2	68.88	+0.06 [−0.01, +0.25]
MOTA	76.8	77.85	+0.08 [−0.03, +0.30]
IDF1	81.4	81.97	+0.16 [−0.02, +0.70]
IDS	–	124	+7 [0, +19]
BoostTrack, online [38]			
HOTA	68.37	68.49	0 (identical)
MOTA	75.56	75.50	0 (identical)
IDF1	81.35	81.41	0 (identical)
IDS	118	113	0 (identical)
BoostTrack, + GBI [38]			
HOTA	71.33	71.72	0 (identical)
MOTA	80.55	81.03	0 (identical)
IDF1	83.84	84.16	0 (identical)
IDS	106	97	0 (identical)
ByteTrack, floor 0.01 [5]			
HOTA	–	67.70	−0.014 [−0.058, +0.009]
MOTA	76.6	77.60	+0.06 [−0.15, +0.34]
IDF1	79.3	79.47	−0.031 [−0.135, +0.035]
IDS	–	214	+4 [−2, +13]
ByteTrack, floor 0.1 [5]			
HOTA	–	67.70	0 (identical)
MOTA	76.6	77.60	0 (identical)
IDF1	79.3	79.47	0 (identical)
IDS	–	214	0 (identical)
OC-SORT, floor 0.01 [6]			
HOTA	66.5	66.43	**+0.61 [+0.36, +1.24]**
MOTA	74.9	74.67	**+1.27 [+0.17, +3.01]**
IDF1	77.7	78.05	+0.66 [−0.25, +1.60]
IDS	–	211	**−8 [−15, −2]**
OC-SORT, floor 0.1 [6]			
HOTA	66.5	66.44	**+0.46 [+0.27, +1.09]**
MOTA	74.9	74.67	**+1.26 [+0.37, +3.34]**
IDF1	77.7	78.05	**+0.287 [+0.003, +0.838]**
IDS	–	213	−14 [−38, +6]

7.3 Results under shifted detector scores

Table 13 and Fig. 8 show the trackers under the four score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA, because no detection reached their thresholds. With the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60. BoostTrack rose from 61.33 to 64.84 under cubing. With temperature $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. The library setting of ByteTrack, whose low thresholds already accept the changed scores, stayed within



	MOT17-02	MOT17-04	MOT17-05	MOT17-09	MOT17-10	MOT17-11	MOT17-13
OC-SORT (0.01)	+0.25	+0.54	+0.45	+2.90	+1.27	+0.34	+0.03
OC-SORT (0.1)	+0.42	+0.21	+0.37	+1.40	+1.14	+0.71	+1.48
PD-SORT	+0.11	+0.20	+1.23	+1.42	+2.39	+0.57	+1.55
SparseTrack	+0.09	0	0	0	+0.52	0	-0.07
ByteTrack (0.01)	0	0	-0.01	0	+0.04	0	-0.21
BoostTrack	0	0	0	0	0	0	0
Hybrid-SORT	0	0	0	0	0	0	0

Fig. 6 HOTA difference, host with SCL minus host alone, in points, for each MOT17 val-half sequence (columns) and each development or predeclared tracker (rows). Each cell prints its value; the shade grows with the size of the difference, and 0 means identical output

0.14 points in all four transforms. Over the 14 conditions, the layer raised HOTA significantly in 7 and lowered it significantly in none. When a transform made a host lose most of its detections, the host also had few identity switches, because it followed few objects; under halved scores it followed none, so it had 0.

Table 12 Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: the tracker alone, without SCL, with the same detections. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better). BoT-SORT: KITTI sequence 0020 is left out of both runs, because the BoT-SORT code stopped with a numerical error in its Kalman update on it; the other 20 sequences are used.

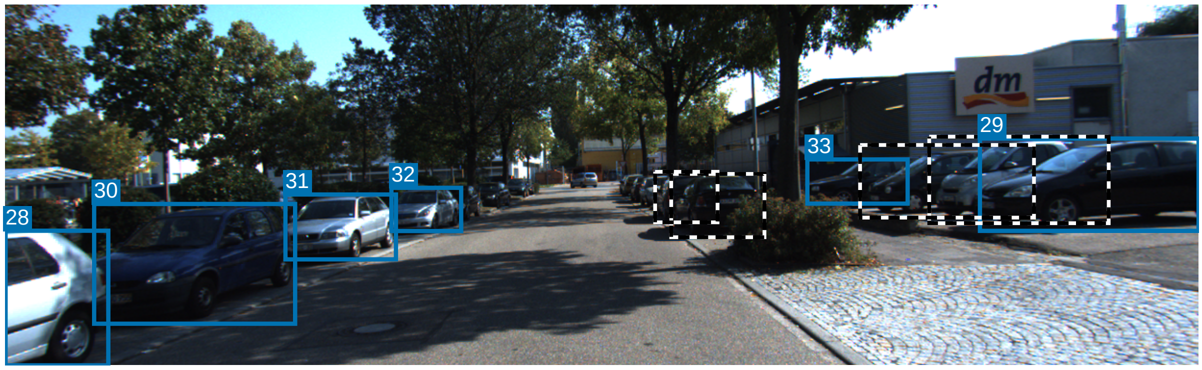
Metric	Host	Δ [95% CI]
ByteTrack [5], *RT-DETR-L*		
HOTA	50.73	**+0.91 [+0.22, +1.55]**
MOTA	47.27	**+5.98 [+1.16, +9.94]**
IDF1	64.98	**+3.87 [+1.67, +5.06]**
IDS	686	**−297 [−530, −122]**
BoT-SORT [39], *RT-DETR-L* (20 sequences)		
HOTA	53.91	+0.61 [−0.39, +1.61]
MOTA	48.25	**+7.86 [+2.21, +13.44]**
IDF1	67.74	**+3.10 [+1.00, +4.57]**
IDS	476	**−226 [−369, −120]**
OC-SORT [6], *YOLOv8n*		
HOTA	36.27	**+7.91 [+5.47, +9.98]**
MOTA	33.99	**+9.40 [+1.10, +12.30]**
IDF1	50.46	**+9.60 [+5.61, +12.16]**
IDS	91	**+79 [+43, +125]**

7.4 Comparison with adaptive-threshold methods

A simpler way to follow the detector scores is to set the tracker's thresholds from them directly. Two such baselines were built on the same input and with the same window of the last 10 frames. They set the tracker's matching and birth thresholds in every frame to the first or the second Otsu split of the recent logits, t_1 or t_2, and pass the scores unchanged, without the mode decision, the new scores, or the rescue. Table 14 compares them with SCL on KITTI, for OC-SORT with YOLOv8n and for both trackers under halved scores. SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the two baselines, and its paired-bootstrap intervals against the t_2 baseline lie above zero in every cell. Adaptive thresholds alone therefore do not explain the gains; the other parts of the layer add to them. With OC-SORT and YOLOv8n, SCL also had fewer identity switches than both baselines (170 against 1,545 and 250). Under halved scores, ByteTrack with SCL had more (501 against 273 and 275), while it missed far fewer objects (FN 7,750 against 12,357 with the t_2 baseline).

Comparison with a published method. ACT [11] sets the high and low thresholds of a two-stage tracker in every frame from the first quartile and the number of the current scores, using fixed score levels. It was run with its published parameters on the same detections as SCL, in three

(a) Tracker alone (OC-SORT): 6 of 10 objects tracked



(b) Tracker with SCL: 9 of 10 objects tracked

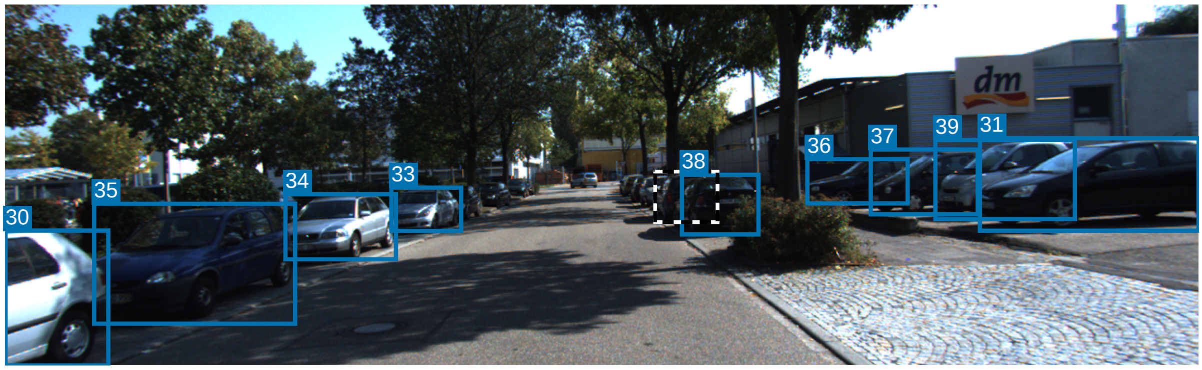


Fig. 7 Example frame from KITTI tracking training [17] (sequence 0001, frame 102), OC-SORT with YOLOv8n detections: (a) the tracker alone, (b) with SCL. Solid boxes with numbers: the tracks output for this frame and their IDs; dashed boxes: labelled cars and pedestrians that no output box covers. The sequence is the one with the median HOTA gain of the 21 sequences, and the frame is the first one in it with the largest gain in tracked objects

settings (Table 15): KITTI with halved scores, for ByteTrack and, with its high threshold as the single threshold, for OC-SORT; its own setting, MOT17 val-half with ByteTrack's official setting and the YOLOX-X detections; and the held-out UAVDT test split with ByteTrack. SCL reached higher HOTA than ACT in every row of Table 15. The difference was significant under halved scores (+2.01 and +10.06 points) and with YOLOv8n on UAVDT (+1.82 points, with +4.00 points of IDF1). In the setting of ACT itself, the two were close (+0.22 and +0.23 points). With RT-DETR-L on UAVDT, SCL reached a MOTA of 27.99 against 5.34 for ACT (difference +22.65, [+9.03, +47.21]): the host produced many false positives, and the fixed levels of ACT removed few of them (178,219 false positives, against 78,975 with SCL). SCL also had fewer identity switches than ACT on MOT17 and UAVDT, significantly with RT-DETR-L on UAVDT (418 fewer). Under halved scores on KITTI it had more (240 and 214 more), because it followed many more objects: it missed 7,750 objects against 12,326 for ACT with ByteTrack, and 13,547 against 21,887 with OC-SORT. The levels of ACT are fixed scores, so they do not move when the scores shift, while the splits of SCL are computed on the scores themselves.

Contribution of each part. Removing one part of the frozen SCL at a time shows where the gains come from, on the KITTI cells. Without the continuation rescue, the HOTA of OC-SORT fell by 3.34 points with YOLOv8n and by 1.16 points under halved scores. Using the clean mode in every frame, that is, without the mode decision, lowered HOTA by 0.90 points for ByteTrack with RT-DETR-L and by 2.40 and 2.44 points for the two trackers under halved scores. Without the

Table 13 Baseline comparison under shifted detector scores on MOT17 val-half. The detector scores are transformed before the tracker, whose published thresholds stay fixed; T is a logit temperature, $\sigma(z/T)$. Host: the tracker alone, without SCL. Δ: with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. IDS: identity switches, a count (lower is better). A host that loses most detections also has few identity switches, because it follows few objects (for example 0 HOTA and 0 IDS under halved scores). Emission floor 0.01 (BoostTrack 0.1).

	HOTA (%)			IDS	
Transform	Host	+ SCL	Δ [95% CI]	Host	+ SCL
ByteTrack, official setting [5]					
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**	48	250
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**	0	298
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**	97	172
$T = 0.5$	67.30	67.29	−0.009 [−0.051, +0.005]	229	229
ByteTrack, library setting [5]					
s^3	66.31	66.31	−0.006 [−0.384, +0.383]	240	265
$0.5\,s$	66.61	66.48	−0.14 [−0.66, +0.14]	297	302
$T = 2$	63.97	63.97	0 (identical)	470	470
$T = 0.5$	66.53	66.52	−0.006 [−0.029, +0.001]	387	388
OC-SORT [6]					
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**	98	235
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**	0	260
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**	171	154
$T = 0.5$	66.61	67.06	+0.45 [−0.01, +0.97]	223	231
BoostTrack, online [38]					
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**	76	166
$T = 2$	67.59	67.59	0 (identical)	119	119

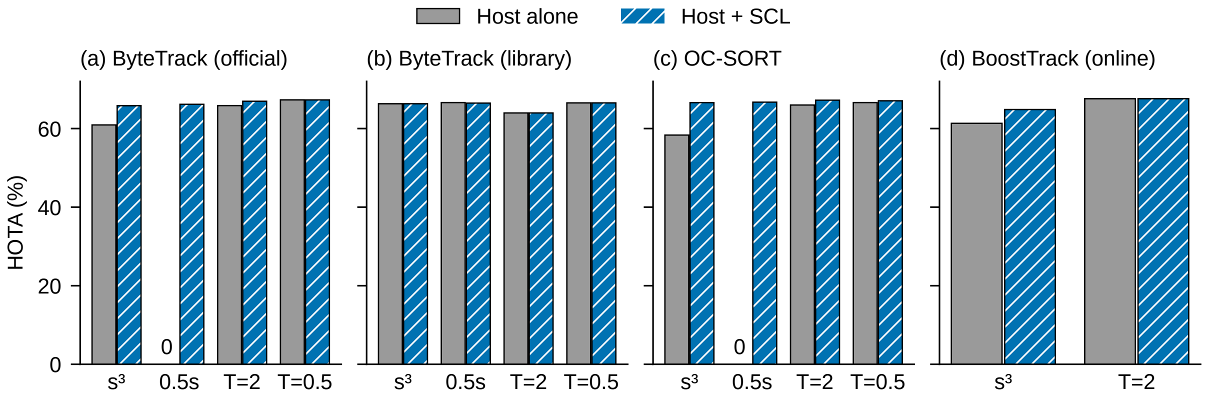


Fig. 8 Shifted detector scores on MOT17 val-half: HOTA of each published tracker with its published thresholds, alone and with SCL, after four monotone transforms of the detector scores (s^3, $0.5\,s$, and temperatures $T = 2$ and $T = 0.5$). (a) ByteTrack, official setting; (b) ByteTrack, library setting; (c) OC-SORT; (d) BoostTrack, online. Plain bars: the tracker alone; hatched bars: with SCL

new scores of the noisy mode, HOTA fell by 1.67 and 0.61 points under halved scores.

7.5 Results on external trackers

Table 16 gives the two predeclared external systems. PD-SORT [40] was reproduced exactly as in the authors' released output. With SCL, its HOTA rose by 0.61 points ($[+0.27, +1.65]$), its MOTA by 1.13 points ($[+0.27, +3.03]$), and its IDF1 by 0.82 points ($[+0.43, +1.94]$), with higher HOTA on 7 of 7 sequences (Fig. 6). FN fell by 1,291, FP rose by 681, and the identity switches stayed almost the same (131 against 129). PD-SORT is a single-stage tracker, and its gain comes from the same step as for OC-SORT; unlike OC-SORT, it was not seen during design. Hybrid-SORT [41], a two-stage tracker, gave identical output, as predicted before the run.

7.6 Results on held-out aerial data

Table 17 gives the held-out UAVDT test split. SCL raised the HOTA of the YOLOv8n stream

Table 14 HOTA and identity switches of SCL and of two simple adaptive-threshold baselines on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: the tracker alone, without any layer. Adaptive t_1, t_2: the tracker's matching and birth thresholds are set in every frame to the first or second Otsu split of the logits of the last 10 frames, and the scores are passed unchanged. Δ: SCL minus the adaptive t_2 baseline, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better); a host with 0 HOTA follows no object, so it also has 0 IDS.

	HOTA (%)					IDS			
Cell	Host	Adapt. t_1	Adapt. t_2	SCL	Δ [95% CI]	Host	Adapt. t_1	Adapt. t_2	SCL
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.17	**+3.09 [+2.00, +4.03]**	91	1545	250	170
ByteTrack, RT-DETR-L, halved scores	49.50	49.58	49.50	51.78	**+2.29 [+0.28, +4.96]**	279	273	275	501
OC-SORT, YOLOv8n, halved scores	0.00	37.68	43.56	46.30	**+2.74 [+1.32, +3.34]**	0	1975	492	308

Table 15 HOTA of ACT [11] and of SCL on the same detections, and the differences SCL minus ACT in HOTA, IDF1, and identity switches (IDS, a count; lower is better), with 95% paired-bootstrap interval below each value; bold intervals exclude zero. ACT: published parameters, no tuning. KITTI: official HOTA, car and pedestrian averaged; MOT17: val-half, ACT's own setting; UAVDT: the protocol of Table 17.

Data	Detector	Tracker	ACT	SCL	ΔHOTA	ΔIDF1	ΔIDS
KITTI, halved scores	RT-DETR-L	ByteTrack	49.77	51.78	**+2.01** **[+0.01, +4.76]**	+1.70 [−1.13, +6.80]	**+240** **[+94, +401]**
KITTI, halved scores	YOLOv8n	OC-SORT	36.24	46.30	**+10.06** **[+6.88, +12.01]**	**+12.03** **[+6.85, +14.69]**	**+214** **[+141, +295]**
MOT17, floor 0.01	YOLOX-X	ByteTrack (official)	67.46	67.68	+0.22 [−0.43, +0.88]	+0.57 [−0.68, +2.12]	−18 [−62, +12]
MOT17, floor 0.1	YOLOX-X	ByteTrack (official)	67.46	67.70	+0.23 [−0.40, +0.90]	+0.60 [−0.55, +2.10]	−22 [−61, +4]
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	**+1.82** **[+0.65, +3.00]**	**+4.00** **[+1.85, +6.22]**	−185 [−480, +13]
UAVDT test	RT-DETR-L	ByteTrack	40.14	42.37	+2.23 [−0.97, +6.32]	+5.35 [−0.03, +12.47]	**−418** **[−718, −123]**

by 1.77 points ([+0.70, +2.94]) and its IDF1 by 3.97 points ([+1.88, +6.17]), and identity switches fell by 45% (from 719 to 393). With RT-DETR-L, the host produced many false positives (MOTA 3.29); with SCL, its MOTA rose by 24.70 points ([+10.06, +51.08]) and its IDF1 by 5.74 points ([+0.13, +13.11]): false positives fell by 58% and identity switches by 56% (from 1,041 to 455). HOTA did not fall with either detector.

8 Conclusion

This paper presented SCL, a control layer that needs no training, adjusts itself from a detector's scores, and adapts a frozen tracker through a declared host contract. The layer splits the logits of recent frames with nested Otsu thresholds and decides whether the stream is clean or noisy. On clean streams it keeps the tracker's settings; on noisy streams it chooses the candidates, assigns them new scores, and sets the thresholds. A continuation rescue gives single-stage trackers the step they lack.

One frozen configuration, developed step by step from AOB, was tested on seven published trackers, four detection sources, and three datasets. It improved single-stage trackers and trackers whose detector or score scale had changed, raised MOTA by 24.7 points and cut identity switches by up to 56% on held-out aerial data, reached higher HOTA than simple adaptive thresholds on the same scores and than the published ACT method under halved scores and on held-out aerial data, kept well-matched two-stage trackers at their own accuracy, showed no statistically significant HOTA loss in any tested cell, and took 2.95 ms per frame on a CPU. These results support placing a self-calibrating layer between the detector and the tracker in systems where the detector changes while the tracker stays the same.

Statements and Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Table 16 Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL. Paper: number reported by the authors (for PD-SORT, their released output). Host: the tracker alone, without SCL (our run of the published tracker). Reproduction class against the authors' number (EXACT: $|\Delta\text{HOTA}| \leq 0.2$; CLOSE: ≤ 1.0 with a documented, untuned cause). Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better); –: not reported.

Metric	Paper	Host	Δ [95% CI]
PD-SORT [40], *EXACT*			
HOTA	68.01	68.01	**+0.61 [+0.27, +1.65]**
MOTA	75.19	75.19	**+1.13 [+0.27, +3.03]**
IDF1	81.03	81.03	**+0.82 [+0.43, +1.94]**
IDS	–	129	+2 [−9, +13]
Hybrid-SORT [41], *CLOSE*			
HOTA	67.1	66.70	0 (identical)
MOTA	75.8	75.52	0 (identical)
IDF1	78.0	77.56	0 (identical)
IDS	–	259	0 (identical)

Table 17 Baseline comparison on the held-out UAVDT test split (20 aerial sequences, not used during design), evaluated once with the frozen SCL. Host: ByteTrack alone, without SCL, with its published setting. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ [95% CI]
ByteTrack, RT-DETR-L [5, 19]		
HOTA	40.07	+2.30 [−1.05, +6.55]
MOTA	3.29	**+24.70 [+10.06, +51.08]**
IDF1	48.59	**+5.74 [+0.13, +13.11]**
IDS	1041	**−586 [−951, −248]**
ByteTrack, YOLOv8n [5, 18]		
HOTA	29.05	**+1.77 [+0.70, +2.94]**
MOTA	16.27	+1.02 [−1.06, +2.86]
IDF1	34.52	**+3.97 [+1.88, +6.17]**
IDS	719	**−326 [−691, −73]**

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval and consent to participate. Not applicable. The study uses only public benchmark datasets and involves no human participants or animals.

Consent for publication. Not applicable.

Data availability. The datasets analysed in this study are publicly available: MOT17 (https://motchallenge.net), KITTI tracking (https://www.cvlibs.net/datasets/kitti), and UAVDT (https://sites.google.com/view/grli-uavdt). The detections released by the authors of the evaluated trackers were obtained from their public repositories. The per-sequence results and bootstrap outputs generated in this study are available from the corresponding author on reasonable request.

Materials availability. Not applicable.

Code availability. The code of the layer, its frozen configuration, the tracker adapters, and the evaluation scripts are available from the corresponding author on reasonable request.

Author contributions. Ahmed Gouda Ismail: conceptualization, methodology, software, investigation, formal analysis, and writing of the original draft. Mohamed S. Mohamed and Tarek Ahmed Mahmoud: supervision, and review and editing of the manuscript. All authors read and approved the final manuscript.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Wu, X., Li, W., Hong, D., Tao, R., Du, Q.: Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. IEEE Geoscience and Remote Sensing Magazine **10**(1), 91–124 (2022) https://doi.org/10.1109/mgrs.2021.3115137

[3] Adžemović, M.: Deep learning-based multi-object tracking: a comprehensive survey from foundations to state-of-the-art. The Visual Computer **42**(9), 368 (2026) https://doi.org/10.1007/s00371-026-04607-7

[4] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp.

3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[5] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[6] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[7] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https://doi.org/10.1109/cvprw50498.2020.00171

[8] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[9] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[10] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[11] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/electronics15174047

[12] Han, Y., Huang, G., Song, S., Yang, L., Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[13] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: Scalable Adaptation of Video Analytics. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, pp. 253–266 (2018). https://doi.org/10.1145/3230543.3230574

[14] Otsu, N.: A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics **9**(1), 62–66 (1979) https://doi.org/10.1109/tsmc.1979.4310076

[15] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016). https://doi.org/10.48550/arXiv.1603.00831

[16] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021). https://doi.org/10.48550/arXiv.2107.08430

[17] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[18] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[19] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-time Object Detection. In: 2024 IEEE/CVF Conference on Computer

Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[20] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q.: The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 375–391. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01249-6_23

[21] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[22] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing **2008**, 1–10 (2008) https://doi.org/10.1155/2008/246309

[23] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-target, Multi-camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[24] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap, (1993). https://doi.org/10.1007/978-1-4899-4541-9

[25] Li, Y., Liu, Y., Zhou, C., Xu, D., Tao, W.: A lightweight scheme of deep appearance extraction for robust online multi-object tracking. The Visual Computer **40**(3), 2049–2065 (2024) https://doi.org/10.1007/s00371-023-02901-2

[26] Bao, Y., Yu, Y., Qi, Y., Wang, Z.: Multiple object tracking with adaptive multi-features fusion and improved learnable graph matching. The Visual Computer **40**(4), 2279–2292 (2024) https://doi.org/10.1007/s00371-023-02916-9

[27] Zhang, Y., Liang, Y., Wang, J., Zhu, H., Wang, Z.: Enhanced multi-object tracking via embedded graph matching and differentiable Sinkhorn assignment: addressing challenges in occlusion and varying object appearances. The Visual Computer **41**(8), 6067–6085 (2025) https://doi.org/10.1007/s00371-024-03772-x

[28] Ding, Y., Wang, Y., Wang, L., Zhang, B., Li, J., Guo, L., Yang, Z., Liang, W.: Enhancing multi-object tracking efficiency through result-guided feature extraction and query filtering. The Visual Computer **41**(11), 8153–8167 (2025) https://doi.org/10.1007/s00371-025-03861-5

[29] Liang, Z., Xiao, G., Hu, J., Wang, J., Ding, C.: MotionTrack: rethinking the motion cue for multiple object tracking in USV videos. The Visual Computer **40**(4), 2761–2773 (2024) https://doi.org/10.1007/s00371-023-02983-y

[30] Sun, Z., Lv, Z., Xiong, M., Hu, W., Wei, G., Zhou, J., He, T., Meng, Q.: Online visual multi-object tracking via blind super-resolution and clustering learning. The Visual Computer **42**(4), 189 (2026) https://doi.org/10.1007/s00371-026-04382-5

[31] Wang, C.-Y., Yeh, I.-H., Liao, H.-Y.M.: YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. In: Computer Vision – ECCV 2024. Lecture Notes in Computer Science, pp. 1–21 (2024). https://doi.org/10.1007/978-3-031-72751-1_1

[32] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-End Object Detection with Transformers. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, pp. 213–229 (2020). https://doi.org/10.1007/978-3-030-58452-8_13

[33] Jung, H., Kang, S., Kim, T., Kim, H.: ConfTrack: Kalman Filter-based

Multi-Person Tracking by Utilizing Confidence Score of Detection Box. In: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 6569–6578 (2024). https://doi.org/10.1109/wacv57701.2024.00645

[34] Solano-Carrillo, E., Sattler, F., Alex, A., Klein, A., Costa, B.P., Rodriguez, A.B., Stoppe, J.: UTrack: Multi-object Tracking with Uncertain Detections. In: Computer Vision – ECCV 2024 Workshops. Lecture Notes in Computer Science, pp. 219–236. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-91585-7_14

[35] Kim, S., Lee, S., Cho, M.: PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 5709–5716 (2026). https://doi.org/10.1609/aaai.v40i7.37491

[36] Reddy, B.S., Chatterji, B.N.: An FFT-based technique for translation, rotation, and scale-invariant image registration. IEEE Transactions on Image Processing **5**(8), 1266–1271 (1996) https://doi.org/10.1109/83.506761

[37] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[38] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: boosting the similarity measure and detection confidence for improved multiple object tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[39] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022). https://doi.org/10.48550/arXiv.2206.14651

[40] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[41] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471